

Article

Hybrid Oversampling and Undersampling Method (HOUM) via Safe-Level SMOTE and Support Vector Machine

Duygu Yilmaz Eroglu ^{1,*} and Mestan Sahin Pir ^{2,3}¹ Department of Industrial Engineering, Bursa Uludağ University, Bursa 16059, Turkey² Graduate School of Natural and Applied Sciences, Industrial Engineering, Bursa Uludağ University, Bursa 16059, Turkey; mestansahin@hotmail.com³ NTT DATA Business Solutions Information Systems Inc., Nidakule Atasehir North Business Center, Begonya Street, No 3/A, Istanbul 34746, Turkey

* Correspondence: duygueroглу@uludag.edu.tr

Abstract: The improvements in collecting and processing data using machine learning algorithms have increased the interest in data mining. This trend has led to the development of real-life decision support systems (DSSs) in diverse areas such as biomedical informatics, fraud detection, natural language processing, face recognition, autonomous vehicles, image processing, and each part of the real production environment. The imbalanced datasets in some of these studies, which result in low performance measures, have highlighted the need for additional efforts to address this issue. The proposed method (HOUM) is used to address the issue of imbalanced datasets for classification problems in this study. The aim of the model is to prevent the overfitting problem caused by oversampling and valuable data loss caused by undersampling in imbalanced data and obtain successful classification results. The HOUM is a hybrid approach that tackles imbalanced class distribution challenges, refines datasets, and improves model robustness. In the first step, majority-class data points that are distant from the decision boundary obtained via SVM are reduced. If the data are not balanced, SLS is employed to augment the minority-class data. This loop continues until the dataset becomes balanced. The main contribution of the proposed method is reproducing informative minority data using SLS and diminishing non-informative majority data using the SVM before applying classification techniques. Firstly, the efficiency of the proposed method, the HOUM, is verified by comparison with the SMOTE, SMOTEENN, and SMOTETomek techniques using eight datasets. Then, the results of the W-SIMO and RusAda algorithms, which were developed for imbalanced datasets, are compared with those of the HOUM. The strength of the HOUM is revealed through this comparison. The proposed HOUM algorithm utilizes a real dataset obtained from a project endorsed by The Scientific and Technical Research Council of Turkey. The collected data include quality control and processing parameters of yarn data. The aim of this project is to prevent yarn breakage errors during the weaving process on looms. This study introduces a decision support system (DSS) designed to prevent yarn breakage during fabric weaving. The high performance of the algorithm may encourage producers to manage yarn flow and enhance the HOUM's efficiency as a DSS.

Keywords: data mining; imbalanced datasets; undersampling; oversampling; classification; decision support systems



Citation: Yilmaz Eroglu, D.; Pir, M.S. Hybrid Oversampling and Undersampling Method (HOUM) via Safe-Level SMOTE and Support Vector Machine. *Appl. Sci.* **2024**, *14*, 10438. <https://doi.org/10.3390/app142210438>

Academic Editors: Gianluigi Ferrari and Eui-Nam Huh

Received: 11 September 2024

Revised: 17 October 2024

Accepted: 11 November 2024

Published: 13 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The recent advancements in data collection technologies have sparked a greater interest in data mining and broadened its range of applications. Additionally, the challenge of identifying rare tags within datasets has contributed to the prevalence of imbalanced datasets. A search for imbalanced datasets in the Web of Science database yielded 7252 documents, including articles, proceedings, early access materials, review articles, and book chapters. In total, 53% of these documents were published between 2021 and 2023.

This rate is an indicator of the increasing interest in and demand for this topic. There is no cutoff value for defining datasets as imbalanced. An imbalanced dataset is one in which some classes are observed more frequently than other classes.

The yarn, the most critical aspect of the fabric weaving process, causes both quality and energy losses when it breaks in the weaving process. In the present study, data were collected to observe the tendency of the yarns to break off during the weaving process. Yarns that are approved in the first quality control process undergo additional operations before transitioning to the weaving stage. Subsequently, efforts were undertaken to forecast whether a possible rupture might occur during the weaving process, prior to its actual occurrence. The developed algorithm (HOUM), utilizing both yarn quality control and production parameters, accurately predicted yarns prone to breakage during the weaving process, achieving a high-performance measure. This stands as the present study's primary achievement.

In the methodology of Weighted Synthetic Informative Minority Oversampling (W-SIMO) [1], the inspiration for the proposed method, informative minority-class data are reproduced around the boundary region of the SVM. However, in our study, non-informative majority-class data, far from the SVM boundary region, were removed from the dataset to avoid the information loss encountered in sample reduction. Then, minority data were reproduced using the safe-level SMOTE (SLS) method [2] if the dataset was still imbalanced.

Essentially, the aim of the HOUM is to address challenges associated with imbalanced datasets by minimizing information loss, improving classifier performance, and employing a combination of techniques to create a balanced representation of classes. This facilitates more effective model learning and prediction. To handle the imbalance problem, the following three steps are applied until the dataset becomes balanced.

- SVM-based undersampling: An SVM model is implemented to find the decision boundary that separates the classes. Majority-class instances that are far from this boundary are identified and considered for undersampling.
- SLS-based oversampling: SLS is applied to generate synthetic instances for the minority class if the dataset is imbalanced.
- Iterative balancing: a loop is created that iterates through the undersampling and oversampling steps until the dataset reaches a balanced state.

Hence, the HOUM employs an iterative method for tackling imbalanced datasets, which integrates SVM-based undersampling and SLS-driven oversampling. Demonstrating the HOUM's success on different datasets and comparing it with existing methods could illustrate its broad applicability. This provides significant insights into the method's overall effectiveness. The ability of the developed algorithm to yield high-performance values when applied to a real dataset in the textile industry (the TexYarn dataset) can also be considered as an indicator of its applicability as a decision support system. These contributions suggest that the method adds substantial value to the literature by addressing classification problems in imbalanced datasets from a different perspective.

Section 2 provides a review of the existing literature. Section 3 explains the preliminaries for classification algorithms, performance metrics, literature datasets used in the paper, and the real production environment's dataset. Section 4 explains the proposed HOUM. The computational results are presented in Section 5. Managerial insights can be found in Section 6. Limitations and future directions of research are discussed in Section 7. Concluding remarks are provided in Section 8.

2. Literature Review

The existing literature on imbalanced data can be examined within certain subgroups, such as pre-processing, algorithmic, and hybrid paradigms, as proposed by Kaur et al. [3]. However, in this section, considering the main frame of the algorithm, the studies on oversampling, undersampling, and hybrid methodologies and studies focusing on DSSs via machine learning in textile production are reviewed.

Random oversampling can be achieved by randomly selecting and increasing minority data and adding them to the original dataset. This method is simple, but exact copies may increase the possibility of overfitting for the datasets that require high oversampling [4]. The most used oversampling method is the SMOTE (Synthetic Minority Oversampling Technique) approach [5]. Unlike random sampling, this method creates synthetic data, analyzing existing minority data. SMOTE cannot reflect the distribution of the original samples in the new artificial samples. Therefore, when using SMOTE-based oversampling methods, there may be errors in the distribution of samples, which may affect the accuracy of the classifier. This will increase the probability and cause the misclassification of samples [6]. Bunkhumpornpat et al. [2] proposed a method called SLS (safe-level SMOTE). The safe level is determined using the nearest neighbor minority samples, and the minority data with the same weight value in the safe-level region are carefully sampled along the line. The authors proved in the study that this method obtained better results than SMOTE. In another study, ESMOTE was proposed as a remedy to the noise problem of SMOTE. In the study, the novel interpolation technique was developed in the production phase of samples, and the selection of beneficial samples was conducted using instance selection based on evolutionary computation [7]. Sáez et al. [8] conducted an oversampling study to address the multiclass imbalance problem and the analysis of class characteristics. In the study, subsets of significant samples could be found in each class and considered by oversampling independently for each. This methodology identifies four different types of samples in multiclass datasets, namely, safe, borderline, rare, and outliers. In both the SIMO and W-SIMO methods, minority examples that are close to the decision boundary, as determined using the SVM, are oversampled [1]. The aim is to reproduce only informative minority data to avoid over-learning while increasing minority data. In the W-SIMO approach, informative minority samples that are misclassified are subjected to a greater degree of oversampling compared to those informative minority samples that are correctly classified. The results of the study were evaluated according to the G-mean criterion, and the method was demonstrated to outperform the commonly used methods such as SMOTE and random oversampling. In another study by Liu et al. [9], a technique based on relative and absolute densities was developed and compared with well-known oversampling methods to resolve imbalances within and between classes.

The random undersampling method is a non-heuristic method that randomly removes data from the majority class until the minority and majority classes reach a reasonable size. This data reduction process may also lead to useful information being discarded during classification. The methods aiming for better data distribution or focusing on data overlap usually provide superior classification performance. In the paper of Vuttipittayamongkol and Elyan [10], the researchers proposed an undersampling technique in binary datasets via the removal of potentially overlapped data points. The method's performance in terms of sensitivity is verified by experiments. Rao et al. [11] addressed the undersampling approach using OPTICS, one of the visualization clustering techniques, to solve the class imbalance problem. The OPTICS clustering technique was incorporated to undersample the majority class. Another study proposed an undersampling method based on the KNN algorithm; samples are removed according to the number of basic neighbors of each class to balance the data [12]. The proposed algorithm was tested on 33 datasets and compared with six methods. Compared with other methods, the results confirmed the validity of the KNN undersampling method.

Recent detailed survey work [13] reviews deep long-tailed learning, where class distributions are imbalanced, with a few classes having many samples and most having very few (a "long tail"). It categorizes methods into three key areas: class re-balancing, information augmentation, and module improvement, while introducing relative accuracy to evaluate how effectively these methods address class imbalance. Although many of the methods discussed in this survey were initially designed for visual applications, they can be adapted to traditional machine learning problems. CReST (A Class-Rebalancing Self-Training Framework for Imbalanced Semi-Supervised Learning) [14] introduces a self-training strategy

that addresses class imbalance in semi-supervised learning by leveraging accurate pseudo-labels, especially for underrepresented classes, to progressively retrain models. CREST+ further enhances this approach with progressive distribution alignment, and both methods outperform traditional semi-supervised learning and rebalancing techniques across different datasets. The FASA (Feature Augmentation and Sampling Adaptation) [15] method creates synthetic features for underrepresented classes using a Gaussian prior and adjusts sampling rates according to the model's classification loss. This adaptive approach prioritizes the augmentation of minority classes, enhancing performance on imbalanced datasets. To gain a more thorough understanding of long-tailed visual recognition, readers may explore additional studies on the topic [16,17].

The aim of hybrid methods, through the use of oversampling and undersampling, is to conquer the class imbalance problem and achieve better performance metrics [3]. In the work of Elyan et al. [18], to handle the class imbalance problem, instances of the majority class were grouped into subclasses via an unsupervised learning algorithm. The proposed class decomposition technique (CDSMOTE) not only reduced the dominance of the majority class but also obstructed information loss. In the study, the oversampling method was applied after the undersampling procedure. Batista et al. [19] proposed two methods, namely, SMOTEENN and SMOTETomek, which work as a combination of over- and undersampling methods. SMOTEENN is a two-step process. First, SMOTE is applied to oversample the minority class. Then, ENN is applied to the resulting dataset to remove instances that are considered noisy or potentially mislabeled. SMOTETomek involves applying SMOTE to oversample the minority class and then using Tomek links to clean the dataset. After SMOTE is applied, Tomek links are identified, and instances involved in these links are removed. RHSBoost [20] uses random subsampling and random oversampling methods with a reinforcement scheme in the developed batch classification method. Based on the experimental results, RHSBoost is a successful classification model for imbalanced data. The RUSBoost algorithm, which is presented by Seiffert et al. [21] for learning from skewed training data, combines boosting and random undersampling. In another hybrid method, RusAda [22], based on RUSBoost, the resampled training dataset is used to build the iteration model. The boosting procedure is used in AdaBoost to improve the algorithm's performance. In another hybrid approach [23], to solve the classification problem of two class imbalanced datasets, firstly, the number of minority samples was increased with SMOTE, and the majority-class samples were reduced with the OSS (one-sided selection) method, and an SVM was used as a classifier.

Yıldırım et al. [24] compiled data mining and machine learning algorithms used in the textile sector. The support vector machine (SVM) was developed by Boser et al. [25] to solve pattern recognition and classification problems, and some studies that utilize the SVM as a machine learning tool are as follows. Hairiness, which affects the quality of yarns, was predicted using an SVM and artificial neural network (ANN) in the study of Vadood et al. [26]. An SVM and ANN were used for image processing in the work of Anami and Elemmi [27]. They focused on classifying fabric images as defective and non-defective. Li and Cheng [28] calculated defect classification accuracy for different types of yarn-dyed fabrics via neural networks and SVMs. The SVM classification scheme was more robust and effective in the study. A proximal SVM was utilized as a classifier to recognize power-loom and handloom fabrics in the study of Ghosh et al. [29]. Studies addressing imbalanced data in textile datasets are rare, but researchers have considered this challenge in their studies in the last few years. Zhan et al. [30] focused on fabric defect classification and proposed a method that provides uniformly distributed samples in each class of training data of an imbalanced dataset. In another study, Haleem et al. [31] proposed an online testing system for yarn quality.

In this study, the proposed method, validated using well-known imbalanced datasets, was applied to a real textile problem. In the problem, after being accepted by the raw material quality department, yarns meeting the required quality values undergo some processes in the production area before weaving. Despite having an accepted quality level and convenient processing parameters, some of these yarns cause quality defects (such

as yarn break-off) in the fabric weaving process. The primary goal of this study was to unearth the possible relationship between yarn acceptance quality parameters, processing parameters, and yarn break-off defects. The DSS (HOUM) demonstrates that detecting and blocking these yarns before they enter the weaving process might increase efficiency in many areas, such as employees, equipment, sustainability, and profitability.

3. Preliminaries

3.1. Classification Techniques Used in the Present Study

3.1.1. K-Nearest Neighbor (KNN)

The K-nearest neighbor algorithm, proposed by Fix and Hodges [32], assigns class labels to unseen data by adopting the most common class label among the unseen data's K-nearest neighbors' classes. The 'K' in K-nearest neighbors represents the number of neighborhoods. A small value of K could lead to over-learning, while a large value might result in generalization. Several criteria can be used to measure distance.

When there are "p" attributes, the distance between points "i" and "j" can be calculated using the Minkowski function, as represented by Equation (1). Notably, this equation calculates the Manhattan distance when "q = 1", the Euclidean distance when "q = 2", and the Chebyshev distance when "q = ∞". In this study, "q" was considered as "2" in order to calculate the Euclidean distance.

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)} \quad (1)$$

3.1.2. Random Forests (RFs)

The random forest algorithm, proposed by Breiman [33], is an amalgamation of tree predictors. Each tree depends on the values of a randomly sampled vector, independently chosen for all trees in the forest. This makes it a supervised classification algorithm. The classification result is ascertained by the majority vote of the decision trees. As the number of trees in the forest increases, the generalization error for the RF algorithm tends towards a limit. The generalization error of an RF classifier is dependent on both the strength of the individual trees within the forest and their interrelation. Internal estimates that keep track of error, strength, and correlation are employed to demonstrate the response to increasing the number of features used in the split. Internal predictors are additionally utilized to measure variable significance. Among the essential features of this algorithm are the ability to work efficiently with large datasets, handle thousands of input variables without necessitating deletion, estimate crucial variables in classification, and incorporate methods to balance errors in datasets with uneven class populations. The main notations used in the random forest algorithm are as follows.

- "S" signifies the training set, a crucial component in machine learning where a model undergoes the learning process.
- "i" essentially represents an index referring to the current decision tree in the processing stage, ranging from 1 to k.
- "k" denotes the quantity of decision trees desired within a random forest.
- "T_i" indicates the i-th decision tree, which is developed through learning from the subset "S_i".
- "S_i" is the subset originating from the initial training set "S", specifically created for the i-th decision tree.

The methodology's algorithm proceeds as follows.

Given a training set, S,

1. For i = 1 to k (number of trees),
 - Build a subset S_i by sampling with replacement from S.
 - Learn a decision tree T_i from S_i.
2. For each node in tree T_i,

- Randomly select a subset of F features.
- Choose the best split from this subset of features.
- 3. Grow each tree to its largest possible size (no pruning is applied).
- 4. Prediction:
 - Make predictions based on the majority vote from the set of k trees.

3.1.3. Support Vector Machine (SVM)

An SVM [25] is a classification algorithm designed to identify the best decision boundary or hyperplane that separates two data classes. The objective is to maximize the margin between the classes, facilitating easier separation. SVMs can be categorized into two groups based on the nature of data separability: linear and non-linear.

Linearly Separable SVM

In this case, a linear hyperplane can be used to separate the data into two classes. The decision boundary for the linearly separable SVM is shown in Figure 1. The following symbols are included in the formulation of a linear SVM.

- $\mathbf{w}$: weight vector in the hyperplane.
- $\mathbf{x}$: input feature vector.
- b : bias scalar, representing the constant term in the hyperplane equation.
- $\|\mathbf{w}\|$: Euclidean norm of the weight vector $\mathbf{w}$.
- Equation (2) represents the decision boundary for the linearly separable SVM:

$$\mathbf{w}\mathbf{x} + b = 0 \quad (2)$$

In a linear SVM, $\mathbf{w}$ is perpendicular to the separating hyperplane, and b helps in positioning this hyperplane in the feature space.

- The goal of the SVM is to maximize the margin, which can be represented in Equation (3)

$$\text{margin} = \frac{2}{\|\mathbf{w}\|} \quad (3)$$

- The decision function ($f(\mathbf{x}_i)$) in Equation (4) assigns class labels based on which side of the hyperplane a data point lies:

$$f(\mathbf{x}_i) = \begin{cases} 1, & \text{if } \mathbf{w}\mathbf{x}_i + b \geq 1 \\ -1, & \text{if } \mathbf{w}\mathbf{x}_i + b \leq -1 \end{cases} \quad (4)$$

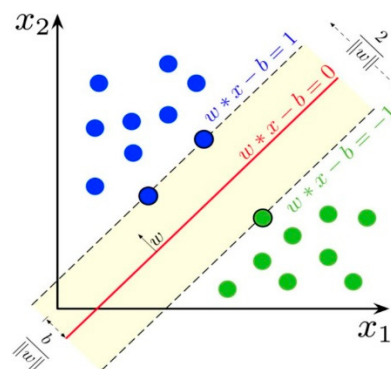


Figure 1. Hyperplane and margins for an SVM for samples with two classes.

In summary, a linear SVM aims to find the optimal hyperplane ($\mathbf{w}\mathbf{x} + b = 0$) that separates classes by maximizing the margin between them. This optimization is achieved

by adjusting the parameters w and b while considering the constraints represented by the class assignment function, as described in Equation (4).

Non-Linearly Separable SVM (Radial)

In most real-world problems, data cannot be linearly separated by a single hyperplane. To solve this problem, the data are mapped to a higher dimensional space, and then a hyperplane is defined there [34]. The following symbols are included in the formulations of a radial SVM.

- $f(x)$: the classification function for input x .
- y_i : class labels.
- a_i : Lagrange multipliers obtained during SVM optimization.
- $\varphi(x_i)\varphi(x_j)$: the dot product in a higher-dimensional space.
- $K(X_i, X_j)$: the kernel function K that computes the inner product without explicitly mapping data into higher dimensions.

In Equation (5), the mapping solution is given by the SVM formula. In a non-linearly separable SVM, the quantities $\varphi(x_i)\varphi(x_j)$ which need to be calculated are scalar products with vital properties. This is called the kernel function (K). When the kernel function is employed, the SVM is formulated as shown in Equation (6). Radial basis and sigmoid kernels are frequently used in studies. These kernels are shown by Equations (7) and (8), respectively [35]. σ in Equation (7) is a parameter that determines the spread of the Gaussian used. δ in Equation (8) pertains to the sigmoid kernel function. This parameter affects the shape of the sigmoid function used for the kernel.

$$f(x) = \text{sgn}\left(\sum_{i=1}^i y_i a_i \varphi(x_i) \varphi(x_j) + b\right) \quad (5)$$

$$f(x) = \text{sgn}\left(\sum_{i=1}^i y_i a_i K(x_i, x_j) + b\right) \quad (6)$$

$$\text{Radial basis : } K(X_i, X_j) = \exp\left(-\frac{1}{2\sigma^2} \|X_i - X_j\|^2\right) \quad (7)$$

$$\text{Sigmoid kernel : } K(X_i, X_j) = \tanh(kX_i X_j - \delta) \quad (8)$$

In this study, both linear and radial basis kernels were tested with the SVM.

3.1.4. Artificial Neural Networks (ANNs)

Artificial neural networks were first proposed by McCulloch and Pitts [36]. The methodology of the artificial neural network is the mathematical modeling of the learning process that imitates the working principle of the human brain. In this way, the algorithm can perform fundamental functions like learning, remembering, and generating new information from existing data.

The backpropagation algorithm [37] can be used for training data. The flow of the procedure is inspired by Haykin's book [38]. The steps of the backpropagation algorithm as well as its index and parameters are summarized as follows:

- p : number of instances ($p = 1 \dots n$).
- m : iteration number.
- A : a small number to update m .
- μ : learning rate parameter, $\mu = A/m$.
- i : number of features (i is 2 in Figure 2).
- h : number of nodes in the hidden layer ($h = 1 \dots H$) (H is 3 in Figure 2).
- $W(i, h)$: weights between input and hidden layer.
- $X(h)$: weights of output from hidden layer.
- $u_p(i)$: input value of i th feature for p th instance.
- b : bias.

○ Y : output value.

1. Initialization of Weights and Biases: Random initial values are assigned between 0 and 0.5 for $W(i, h)$, $X(h)$, and b . This step is crucial as the network needs proper starting values to begin the training process.

2. Forward Propagation:

- Calculating $Vp'(h)$: $Vp'(h)$ is calculated for each hidden node and instance using Equation (9), which involves the weighted sum of inputs to the hidden nodes.

$$Vp'(h) = \sum_{i=1}^I W(i, h) * u_p(i) \quad (9)$$

- Applying the Logistic Function: Using the logistic function in Equation (10), $Vp(h)$ is calculated for each instance and hidden node. The logistic function transforms the weighted sum into an output between 0 and 1, which is commonly used in neural networks for binary classification tasks.

$$Vp(h) = \frac{1}{1 + e^{-Vp'(h)}} \quad (10)$$

3. Calculating the Predicted Output: the predicted output value Op is computed for each instance using Equation (11), which involves the bias term and the weighted sum of hidden node outputs.

$$Op = b + \sum_{h=1}^H X(h)Vp(h) \quad (11)$$

4. Backpropagation (Updating Weights and Biases):

- Updating the Bias (b): the bias term is adjusted based on the error between the predicted output and the actual output using Equation (12).
- Updating $W(i, h)$ Weights: the weights between the input and hidden layer $W(i, h)$ are updated using Equation (13).
- Updating $X(h)$ Weights: the weights between the hidden and output layer $X(h)$ are updated using Equation (14).

$$b = b + \mu \sum_{p=1}^n (Yp - Op) \quad (12)$$

$$W(i, h) = W(i, h) + \mu \sum_{p=1}^n (Yp - Op) X(h) Vp(h) (1 - Vp(h)) u_p(i) \quad (13)$$

$$X(h) = X(h) + \mu \sum_{p=1}^n (Yp - Op) Vp(h) \quad (14)$$

5. Error Computation: calculating the sum squared error (SSE), which is computed using Equation (15) to evaluate the network's performance.

$$SSE = \sum_{p=1}^n (Yp - Op)^2 \quad (15)$$

6. Iteration and Termination: Steps 2–5 are repeated until a termination criterion is met, for example, when a small SSE value is achieved or the maximum number of iterations is reached. This iterative process fine-tunes the weights and biases of the network to minimize the error between predicted and actual outputs.

This sequence aligns with the iterative nature of the backpropagation algorithm. It trains neural networks by updating weights and biases based on the calculated errors until the network learns to make accurate predictions.

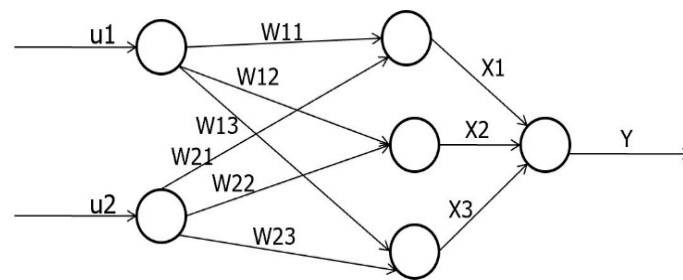


Figure 2. Network graph of ANN with one hidden layer and one output neuron.

3.2. Performance Metrics

Model selection and model evaluation are two essential processes in machine learning. Therefore, performance measures serve as critical indicators for assessing a classifier's effectiveness and steering its learning process [39]. In classification problems, accuracy is generally used as an evaluation criterion; however, it should not be used as the only criterion in imbalanced datasets. In a dataset with 10% minority and 90% majority, even if all minority classes are predicted incorrectly, the accuracy will be 90%. Thus, further clarification of this ratio may be necessary to truly compute the model's success.

The complexity matrix is commonly used for evaluation in classification problems (an example is shown in Table 1). The abbreviations in Table 1 represent the following: TP is the number of correctly classified samples belonging to the positive class, TN is the number of correctly classified samples in the negative class, FP is the number of misclassified samples in the negative class, and FN is the number of misclassified samples in the positive class. Using these basic definitions, the following metrics can be calculated to compare the performance of different algorithms.

Table 1. Complexity matrix for the example with 2 class labels.

	Predicted as Positive	Predicted as Negative
Actual positive class	TP	FN
Actual negative class	FP	TN

Accuracy is the measure of the number of correctly predicted samples among all samples for any classification model. The accuracy rate is shown in Equation (16).

$$\text{Accuracy Rate} = \frac{TP + TN}{TP + FN + FP + TN} \quad (16)$$

Recall is the measure of positive samples accurately predicted by the model. It is calculated in Equation (17). It is sometimes called the true positive rate (TPR) or sensitivity.

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (17)$$

Specificity is a measure of negative samples accurately predicted by a model. It is also sometimes referred to as the true negative rate (TNR). It is calculated via Equation (18).

$$\text{Specificity (True Negative Rate)} = \frac{TN}{TN + FP} \quad (18)$$

Precision is defined as the ratio of true positives (TPs) to the total number of positive samples predicted. It is calculated in Equation (19).

$$\text{Precision} = \frac{TP}{TP + FP} \quad (19)$$

The F-score evaluates both the recall and precision and is calculated in Equation (20). The F-score can be interpreted as the harmonic mean of the recall and precision.

$$F - Score = \frac{2 \times Recall \times Precision}{Recall + Precision} \quad (20)$$

G-mean considers both positive class and negative class performance and uses the geometric mean to combine them. It is calculated in Equation (21). A high G-mean value can be obtained when the algorithm has high prediction accuracy for both the positive class and negative class.

$$G - Mean = \sqrt{\frac{TP}{TP + FN} \times \frac{TN}{TN + FP}} \quad (21)$$

3.3. Compared Solution Methods for Imbalanced Datasets from the Literature

The SMOTE (Synthetic Minority Oversampling Technique) approach [5] is one of the most popular methods for balancing datasets. This method creates synthetic data by analyzing existing minority data. SMOTE does not accurately reflect the distribution of the original samples in the newly created artificial samples. Therefore, when using SMOTE-based oversampling methods, there may be errors in the distribution of samples, which may affect the accuracy of the classifier. This may increase the probability and cause misclassification of samples [6].

The SLS (safe-level SMOTE) method was proposed by Bunkhumpornpat et al. [2]. The “safe level” is determined using nearest neighbor minority samples, and minority data with the same weight value within this safe-level region are carefully sampled along the line. The authors proved in the study that this method obtained better results than SMOTE.

The combination of SMOTE and ENN and SMOTE and Tomek represents hybrid resampling techniques designed to solve class imbalance in datasets. Both methods commence with the oversampling step of SMOTE, generating synthetic instances for the minority class. Following this, SMOTEENN employs edited nearest neighbors to eliminate examples, while SMOTETomek utilizes Tomek links to identify and remove instances causing ambiguity at the decision boundary. In this study, SMOTEENN and SMOTETomek were utilized to compare the proposed method [19].

In another study, Piri et al. [1] proposed SIMO (Synthetic Informative Minority Oversampling) and a variation known as weighted SIMO (W-SIMO). In these algorithms, after separating the training and test data, an SVM is applied to the training data, and the decision boundaries between the classes can be determined. The aim of these methods is to reproduce only informative minority data close to the boundary region to avoid overfitting when oversampling minority data. The results of the study were verified with different imbalanced data learning approaches using the G-mean metric.

Before briefly introducing RusAda [22], we should note that boosting [40] is an idea that focuses on misclassified instances and gives more weight to these instances. The AdaBoost algorithm [41] can be considered the first boosting algorithm. It combines boosting and a random undersampling strategy, outperforming SMOTEBoost. RusAda, a method we compared our results with, is an algorithm that incorporates the boosting procedure from AdaBoost to improve the existing RusBoost algorithm.

3.4. Datasets for Validation

The literature datasets used within the scope of the present study were obtained from Kaggle and UCI data repositories. The “TexYarn” dataset was collected directly from the production plant.

3.4.1. Datasets from the Literature

The benchmark datasets were chosen based on their use in similar studies found in literature reviews; this allows for a more direct comparison. The general information of these datasets is displayed in Table 2. Brief details about the TexYarn dataset arranged in

another study that will be explained in the next subsection are also appended to the final line of Table 2.

Table 2. Characteristics of benchmark datasets.

Datasets	# of Features	# of Instances	# of Instances in Majority Class (Np)	# of Instances in Minority Class (Nn)	Imbalance Rate (Np/Nn)
Climate	21	540	494	46	10.74
Diabetes	9	768	500	268	1.87
Liver	11	583	416	167	2.49
Haberman	4	305	224	81	2.77
Transfusion	5	748	570	178	3.20
Ionosphere	32	351	225	126	1.79
Column_2c	7	310	210	100	2.10
TexYarn	8	979	937	42	22.31

3.4.2. TexYarn Dataset for Weaving

Before the weaving process, the necessary yarns undergo a sequence of control and production stages until they reach the looms. During the incoming material control phase, the yarn can either be accepted, rejected, or conditionally approved. If accepted, the yarn may be subject to a series of production stages (for example, fixing) in order to meet the requisite fabric specifications. Incoming material control test parameters and production process parameters vary based on the type of yarn required by the plant. With these parameters, the aim is to automatically predict which yarn may break during the weaving process. This will help to formulate the decision to either reject or approve the lot prior to its entry into the weaving looms. The dataset features are described below:

- The required tests are applied to yarn lots, which are obtained from the supplier during the quality control phase, and only yarns with values within specified ranges are allowed into the production area. The test parameters include Boiling Shrinkage, Breaking Load, Strength, Denier, and Elongation.
 - Boiling Shrinkage: accepted values are between 1.40 and 67.00.
 - Breaking Load: accepted values are between 124.19 and 2329.20.
 - Strength: accepted values are between 1.40 and 4.85.
 - Denier: accepted values are between 30 and 673.
 - Elongation: accepted values are between 14.00 and 221.84.
- Production parameters include the process parameters applied to the yarn lots following the incoming control process, such as Waiting Duration and Temperature.
 - Waiting Duration: applied values are between 30 and 50.
 - Temperature: applied values are between 80 and 122.

The goal of this dataset is to predict whether the yarn will break during the weaving process based on these seven features. The proposed decision support system aims to enhance productivity by preventing yarns which are liable to break from being used in the looms.

4. Proposed Algorithm: HOUM

This study introduces the Hybrid Oversampling and Undersampling Method (HOUM), which uses SLS for oversampling and an SVM for undersampling with imbalanced datasets. In this proposed method, undersampling is applied to the majority-class data that are far from the decision boundary determined using the SVM. Subsequently, if the dataset remains imbalanced, SLS performs oversampling. The procedure is continued until the dataset becomes balanced. The goal is to prevent the loss of valuable information by removing data from regions that are far from the decision boundary. In the SIMO method, which inspired our study, oversampling is only applied to the minority data located in the boundary region that is identified with the SVM.

This work was structured around the following assumptions: Obtaining balanced datasets from binary class imbalanced datasets may increase the classification performance. The non-decisive data of the majority class are far from the decision boundary obtained via the SVM. Therefore, reducing instances that are far from the boundary may contribute to balancing the data without information loss. SLS is another tool utilized to balance the data by increasing the amount of minority data. The G-mean value was chosen as the performance indicator due to its high prediction accuracy for both positive and negative classes.

The main procedure of the HOUM is shown in Figure 3.

- An SVM is applied to the imbalanced training data as shown in Figure 3A.
- The red points in Figure 3B indicate the detected majority data furthest from the decision boundary.
- Sample reduction is applied, as shown in Figure 3C.
- Oversampling is performed via SLS, as shown in Figure 3D.

Figure 4 displays a flowchart of the methodology. According to Figure 4, initially, the dataset is normalized. The data are then divided into a training set and a test set. The training set is used to train the classifier, while the test set is utilized to evaluate the classifier's performance. If the data are imbalanced, then undersampling and oversampling methodologies can be applied to balance the data. After balancing the data, a classification algorithm is used. Finally, the classifier is evaluated using the test set, and performance metrics are calculated.

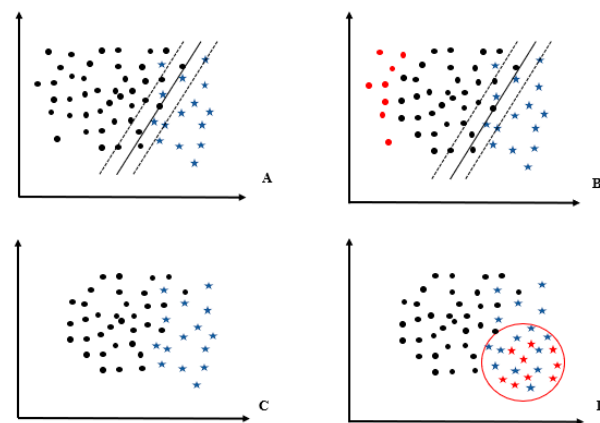


Figure 3. The main procedure of the HOUM: (A) SVM implementation. (B) Selecting majority data far from the decision limit. (C) Performing instance reduction. (D) Oversampling via SLS. Black dots: majority data; blue stars: minority data; red dots: data selected from the majority class for removal; red stars: oversampled minority data.

The steps of the HOUM are as follows.

1. Values of “0” and “1” are assigned to class labels. The value of “1” is assigned to the minority-class labels.

2. The min–max normalization procedure is applied to all features, excluding the class label, using Equation (22) [38]. The definitions of notations are as follows.

v : value for feature that will be normalized.

v' : normalized value.

$\min_A$: minimum value for Feature A.

$\max_A$: maximum value for Feature A.

new_min_A : minimum value after normalization.

new_max_A : maximum value after normalization.

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A \quad (22)$$

3. The data are divided into 80% for training and 20% for testing. (Note: oversampling and undersampling operations are applied only to the training data.)
4. The balance of the training dataset is determined (a balanced condition indicates that the class distribution is within $50 \pm 5\%$). If this condition is met, the process proceeds to step 8.
5. Undersampling is applied to the majority of the data far from the decision boundary as follows.
 - The first quartile (25th percentile) of decision values is considered to belong to the majority class.
 - The aforementioned first 25th percentile is removed from the dataset.
6. The balance of the training dataset is determined. If the dataset is balanced, the process proceeds to step 8.
7. Oversampling is applied using SLS. Then, the process returns to step 4.
8. If the data are balanced, they can be classified using classification algorithms.
9. The classifiers are evaluated with the test data, and the results are interpreted using accuracy and G-mean metrics.

The complexity analysis of the HOUM algorithm consists of three main steps: SVM-based undersampling, SLS-based oversampling, and iterative balancing. In the first step, SVM-based undersampling, the training of the SVM in HOUM-R with a radial kernel, typically has a complexity between $O(n^2)$ and $O(n^3)$ as it involves calculating the kernel matrix, where n represents the total number of samples in the dataset [42]. In the second step, the SLS method determines the safe levels of the minority-class samples using the nearest neighbors and has a complexity of $O(n \cdot k \cdot d)$, where n is the number of data points, k is the number of neighbors, and d is the number of features [2]. In the iterative balancing step, these two processes are repeated until the dataset is balanced, leading to an overall complexity of $O(I \cdot (n^2 + n \cdot k \cdot d))$, where I represents the number of iterations.

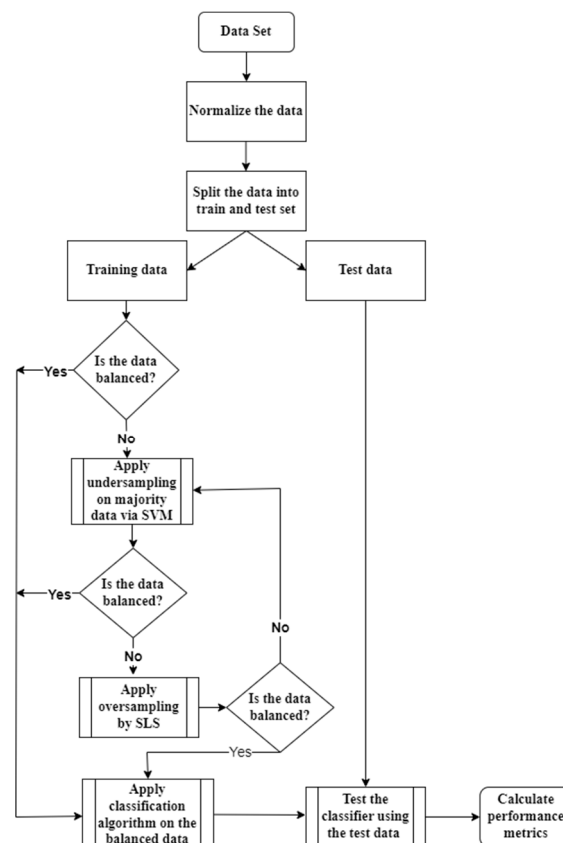


Figure 4. Flowchart of the proposed HOUM algorithm.

5. Computational Results

The proposed HOUM algorithm was run using both a linear SVM (HOUM-L) and a radial SVM (HOUM-R). In the present study, KNN, RF, SVM, and ANN classifiers were used. In the first stage of our research, we balanced our datasets using the developed techniques HOUM-R and HOUM-L. Additionally, the well-known SMOTE and hybrid resampling methods such as SMOTEENN and SMOTETomek were utilized to compare the proposed approach. These balanced datasets, along with an imbalanced version referred to as the “Original Dataset,” were analyzed across stated classification algorithms. In the second stage, we examined the contribution of the proposed method to the literature by comparison with similar studies such as W-SIMO and RusAda.

In the application, the “R i386 4.0.3” version of the R statistical software development and data analysis program was used [43,44]. The packages and parameter values used are summarized in Table 3.

In the present study, sample reduction and augmentation operations were only applied to the training data. The test data underwent no processing. In the hybrid method’s implementation, the “e1071” package was used for the SVM function during the sample reduction phase, deriving data for both radial and linear kernels. The decision variable values (decision.values) were determined via the SVM, and instances before the first quarter of the majority data class’s decision values were removed from the dataset. Then, the volume of minority-class data was augmented using the SLS function from the “smotefamily” package, employing the safe-level SMOTE method.

Table 3. Algorithms and parameters used.

Algorithm	R Package	Parameters
KNN	class	k = 1:20; preProc = “center”, “scale”
RF	randomForest	mtry = 1:10; method = ‘rf’; metric = ‘Accuracy’
SVM	e1071	kernel = Radial/Linear; sigma = 0.01, 0.015; C = 0.75, 1, 1.25
ANN	nnet	decay = 0.001, 0.01, 0.1; size = 1:10
SLS	smotefamily	K = 5; C = 5

5.1. Evaluation of Balancing Methods: A Focus on HOUM Variants

Table 4 summarizes the accuracy and G-mean results of the algorithms. Five balancing methods (HOUM-R, HOUM-L, SMOTE, SMOTEENN, and SMOTETomek) were applied (to balance the datasets) to each imbalanced dataset (Climate, Diabetes, Liver, Haberman, Transfusion, Ionosphere, Column_2c, and TexYarn). The original forms of the datasets were also studied, as shown in the last columns of Table 4. After balancing the datasets using the five balancing methods, the KNN, random forest (RF), support vector machine (SVM), and artificial neural network (ANN) algorithms were applied to compare the accuracy and G-mean parameter values. The best accuracy and G-mean results are written in bold and italics for each line. According to the results, in five of the eight datasets (Liver, Haberman, Transfusion, Ionosphere, and TexYarn), the highest G-mean results were obtained after applying one of the proposed methods (either HOUM-R or HOUM-L). The values are underlined in Table 4. The Wilcoxon signed rank test was performed on five datasets (Liver, Haberman, Transfusion, Ionosphere, TexYarn), and the *p*-value was found to be 0.0625 when the variants of the HOUM were compared to the algorithm that gave the best G-mean value (considering same classification technique). Therefore, the difference between the values of compared algorithms is large enough to be statistically significant, with a 93% confidence interval in the analyses. When the results of the proposed method were thoroughly examined, three of the five best G-mean results (Liver, Haberman, Transfusion) were recorded for the ANN classifier, and two of them (Ionosphere, TexYarn) were found for the RF classifier. This could demonstrate the suitability of the proposed method for these two classifiers. Please also note that, according to the TexYarn dataset, almost all the

yarn lots can be detected, which might suggest the use of the proposed methodology as a decision support system (DSS) to increase efficiency.

Hybrid Oversampling and Undersampling Method (HOUM), via SLS and SVM.

Table 4. Comparing the performance of variants of the HOUM, SMOTE, SMOTEENN, and SMOTE-Tomek techniques and the original dataset.

Dataset	Classifier	HOUM-R		HOUM-L		SMOTE		SMOTEENN		SMOTETomek		Original Dataset	
		Accuracy (%)	G-Mean (%)	Accuracy (%)	G-Mean (%)	Accuracy (%)	G-Mean (%)	Accuracy (%)	G-Mean (%)	Accuracy (%)	G-Mean (%)	Accuracy (%)	G-Mean (%)
Climate	KNN	82.00	50.20	81.00	53.50	76.00	65.60	67.59	72.84	75.93	56.42	92.00	33.30
	RF	91.00	33.20	91.00	0.00	93.00	47.10	76.85	78.24	88.88	61.27	90.00	0.00
	SVM	95.00	80.80	94.00	73.80	91.00	72.60	75.92	77.72	81.48	76.06	91.00	0.00
Diabetes	ANN	95.00	74.20	94.00	73.80	96.00	87.30	90.74	0.00	90.74	0.00	95.00	66.70
	KNN	70.00	69.40	67.00	65.30	70.00	67.90	62.34	64.69	64.28	65.83	73.00	61.10
	RF	72.00	70.30	70.00	66.40	71.00	69.90	71.43	73.29	75.97	76.06	71.00	65.40
Liver	SVM	74.00	71.00	71.00	70.70	71.00	71.30	65.58	67.68	70.13	70.30	75.00	67.30
	ANN	75.00	72.70	72.00	68.40	69.00	69.90	70.13	69.45	70.13	69.46	74.00	46.40
	KNN	60.00	57.40	68.00	57.60	65.00	60.50	58.62	60.58	62.07	59.20	69.00	50.90
Haberman	RF	75.00	64.90	75.00	62.80	73.00	60.20	68.10	68.43	66.38	53.00	72.00	17.40
	SVM	64.00	68.20	71.00	70.00	66.00	69.60	66.38	68.71	66.38	68.71	71.00	46.30
	ANN	69.00	73.60	71.00	67.80	67.00	70.30	61.20	0.00	61.20	0.00	64.00	57.30
Transfusion	KNN	63.00	55.50	66.00	53.80	63.00	63.10	62.90	58.39	61.29	52.34	73.00	47.70
	RF	65.00	53.00	71.00	56.20	66.00	53.80	66.13	54.82	66.13	43.23	75.00	57.70
	SVM	76.00	58.40	73.00	47.70	78.00	63.10	66.13	57.74	69.35	52.94	73.00	0.00
Ionosphere	ANN	73.00	64.80	75.00	70.50	68.00	70.30	67.70	38.43	67.74	38.44	75.00	53.30
	KNN	68.00	61.60	67.00	58.00	65.00	59.70	65.33	62.19	73.33	61.49	79.00	56.70
	RF	75.00	64.00	72.00	67.70	73.00	65.80	68.00	63.78	66.67	56.69	80.00	54.80
Column_2c	SVM	79.00	71.80	73.00	70.40	67.00	71.80	69.33	69.65	66.67	68.66	77.00	16.90
	ANN	79.00	72.80	77.00	73.20	73.00	66.80	70.66	64.13	70.67	64.14	78.00	37.50
	KNN	92.00	89.40	92.00	89.40	92.00	89.40	87.32	82.38	90.14	86.60	87.00	80.00
TexYarn	RF	97.00	95.90	94.00	94.70	95.00	94.80	92.96	91.50	94.37	93.39	95.00	94.80
	SVM	95.00	93.80	90.00	90.40	95.00	93.80	83.10	75.59	85.92	81.41	95.00	93.80
	ANN	87.00	81.50	87.00	81.50	91.00	87.20	90.14	86.60	90.14	86.60	88.00	82.50
TexYarn	KNN	79.00	76.40	72.00	71.90	77.00	73.60	80.65	85.28	82.26	84.09	80.00	80.50
	RF	82.00	83.00	82.00	83.00	80.00	80.50	85.48	87.90	79.03	72.65	83.00	84.20
	SVM	72.00	74.30	77.00	79.20	70.00	74.60	83.87	87.90	85.48	86.46	77.00	73.60
TexYarn	ANN	75.00	75.60	77.00	79.20	74.00	74.40	82.26	76.87	82.26	76.87	77.00	71.70
	KNN	98.00	86.40	98.00	86.40	98.00	86.40	88.82	98.00	88.82	98.00	98.00	86.40
	RF	99.00	99.99	99.00	99.99	99.00	93.50	98.11	97.00	97.00	97.00	99.00	93.50
TexYarn	SVM	97.00	98.90	98.00	99.20	97.00	98.70	98.98	71.26	97.00	98.70	98.00	79.10
	ANN	98.00	99.50	98.00	99.20	97.00	98.70	97.00	0.00	97.00	0.00	95.00	0.00

Bold and italics: best accuracy and G-mean per row; underlined: best G-mean for each dataset.

5.2. Comparing HOUM with W-SIMO and RusAda

In this subsection, the best G-mean values of the HOUM techniques are compared with the G-mean results of W-SIMO [1] and RusAda [22] methodologies, which are proposed to balance the dataset. Table 5 demonstrates the performance of the HOUM, W-SIMO, and RusAda methods. The * sign in some cells indicates the absence of certain results due to the focus on different datasets in various studies. Please consider the following points when comparing the algorithms.

- The G-mean values in the first column of Table 5: All the datasets were balanced using the HOUM (HOUM-R was used for Liver and Ionosphere, and HOUM-L was used for Haberman and Transfusion). Then, the Diabetes, Liver, Haberman, and Transfusion datasets were classified using an ANN, while the Ionosphere dataset was classified using the RF method.
- The G-mean values in the second column of Table 5: All the datasets were balanced using W-SIMO. Then, all datasets were classified using an SVM.
- The G-mean values in the third column of Table 5: All the datasets were balanced using RusAda. Then, all datasets were classified using a decision tree.

The utilized classifier might be an important indicator for this comparison. Here, the compatibility of the classifier with the balancing method can be considered a significant factor. The developed HOUM algorithm has demonstrated high performance with an ANN, W-SIMO shows good compatibility with SVMs, and RusAda can be effectively employed with decision trees.

According to Table 5, the developed HOUM algorithm obtained better performance results in four of five datasets. However, the conclusive comparison results were obtained after classifying the datasets using the ANN and RF algorithms after balancing W-SIMO and RusAda.

Table 5. Comparing the performance of various methods with common datasets.

Dataset	HOUM	W-SIMO	RusAda
Diabetes	72.70%	76.26%	76.15%
Liver	73.60%	69.08%	*
Haberman	70.50%	*	64.79%
Transfusion	73.20%	*	69.21%
Ionosphere	95.90%	94.11%	90.14%

Bold numbers: highest performance per dataset across methods. *: result not available for the relevant dataset.

6. Managerial Insights

The problem of class imbalance arises when the number of instances representing one class is significantly lower than those of the other classes. Such datasets have garnered considerable attention from researchers and practitioners due to the prevalence of real-world applications where the collected raw data meet this criterion. However, these imbalanced datasets typically lead to lower performance indicators for classification techniques. Researchers are focused not only on developing new algorithms to balance these datasets but also on applying novel techniques to potential implementation areas to enhance efficiency, thereby saving time and resources. In this study, the developed algorithm was first validated via comparison with algorithms designed for the same purpose, as outlined in the literature. Subsequently, the algorithm was applied to a real-life problem in the textile industry, and the results were interpreted.

In the textile industry, the main raw material for fabric is yarn. Yarn undergoes various chemical and physical processes to acquire the desired properties. Yarn is defined and differentiated by many characteristics, such as filament value, density, twist value, thickness, and color. Woven fabrics reach their initial fabric form after the weft and warp yarns are woven on looms with a specific weave. Stoppages in looms due to weft and warp breakage can lead to significant defects in the fabric. Defects in weft lots (since the same lot is transferred to different bobbins to be used on different machines) lead to stoppages in more than one machine due to breakage errors. If these defects occur in warp lots, longitudinal breaks will occur, causing faults in meters and leading to machine stoppages. For example, a breakage of 10 g can result in more than 100 kg of defective fabric in some cases.

Therefore, companies that closely monitor technological advancements and are capable of automatically collecting data can achieve high efficiency gains by integrating such decision support systems into their existing structures.

7. Limitations and Future Directions

In this study, the G-mean metric, which considers all elements of the confusion matrix, was prioritized. Additionally, the F-Score values for all datasets were calculated using the unprocessed original versions of the datasets, SMOTE-based methods, and the proposed algorithm. Although the original dataset achieved the highest F-Score in four out of the eight datasets, its G-mean values were significantly lower, indicating poor overall performance. As a result, improving the G-mean became the primary focus for comparison and further enhancement.

Based on previous studies [45], SMOTETomek was shown to have a complexity of $O(T.k.d) + O(n^2.d)$, and SMOTEENN was shown to have a complexity of $O(T.k.d) + O(n.k.d)$ where T is the number of synthetic samples, k is the number of neighbors, n is the number of data points, and d is the dimensionality. HOUM-R, which incorporates SVM-based undersampling and SLS for oversampling, has a higher complexity of $O(I.(n^2 + n.k.d))$

where I is the number of iterations required to achieve balance. Despite the HOUM's increased computational cost, it offers a significant advantage by focusing on the decision boundary through the SVM, thus generating more informative samples and reducing the risk of overfitting, which can enhance classification performance in imbalanced datasets. This trade-off between computational cost and improved performance makes the HOUM a valuable approach in scenarios where the metric of G-mean is a critical concern.

While the Hybrid Oversampling and Undersampling Method (HOUM) shows promising results in balancing binary class datasets, several limitations need to be addressed. The method's performance on multiclass imbalanced datasets remains unexplored, posing challenges in adapting it to more complex problems. Additionally, the HOUM's reliance on computationally expensive techniques like SVMs for undersampling can hinder its scalability when applied to large datasets. The method's efficiency may also vary depending on the classifiers used, with its adaptability across a broader range of algorithms yet to be thoroughly tested. Furthermore, the use of fixed parameters, which were not optimized across different datasets, could limit the method's generalization, requiring manual fine-tuning to achieve optimal performance in new domains. Lastly, the sequential nature of the oversampling and undersampling steps may risk overfitting or data loss, particularly in highly imbalanced datasets.

To address these challenges, future research should focus on extending the HOUM to multiclass problems, optimizing parameters through automated techniques, and improving computational efficiency to make the method more scalable. Exploring hybrid models that integrate resampling with classification in a unified framework could also mitigate potential risks of overfitting or data loss. By addressing these limitations, the HOUM can be adapted for a wider range of real-world applications and larger, more complex datasets.

8. Conclusions

Imbalanced datasets were examined within the scope of the present study. The literature review revealed the importance of augmenting minority-class samples. It also highlighted the challenges created by reducing valuable data from the majority class. To facilitate comparison with the existing literature, the frequently utilized G-mean criterion was chosen as the performance evaluation metric. In this study, the majority-class data that may not provide information in the area distant to the decision boundary found by the SVM were reduced to balance the classes. If a balanced class was not achieved, the data of the minority classes were increased using SLS. These processes were cycled repeatedly until a balanced class was reached. Once the balanced dataset was achieved, some well-known classification algorithms (KNN, RF, SVM, and ANN) using datasets from similar studies in the literature were required in order to validate the developed methodology. The proposed HOUM algorithm was compared with the SMOTE, SMOTEENN, and SMOTETomek algorithms and the original dataset for validation. In five of eight datasets, variations of the HOUM obtained better performance metrics. The HOUM was also compared with two methodologies, W-SIMO and RusAda, utilizing common datasets, and the successful results validated the algorithm's performance. The aim of the DSS project, conducted in the weaving industry, was to bridge academic research with a real production environment. The purpose of the aforementioned project was to prevent yarn breakage during the weaving process on looms. One process was selected to achieve the objective, and the most important incoming material control parameters and production parameters were automatically collected; these data are called the TexYarn dataset. When the HOUM was applied to this dataset, nearly all yarns in the test set could be detected.

Author Contributions: M.S.P. developed and coded the algorithms introduced in the study. D.Y.E. supervised the project and made important contributions to the study, the algorithms, and the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets utilized in this study are available in UCI data repositories [46].

Acknowledgments: The real dataset, TexYarn, used in this study, was gathered as part of a project supported by Güncel Yazılım R&D Center and The Scientific and Technical Research Council of Turkey (TUBITAK-TEYDEB-3190654).

Conflicts of Interest: Author Mestan Sahin Pir was employed by the company NTT DATA Business Solutions Information Systems Inc. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Piri, S.; Delen, D.; Liu, T. A synthetic informative minority over-sampling (SIMO) algorithm leveraging support vector machine to enhance learning from imbalanced datasets. *Decis. Support Syst.* **2018**, *106*, 15–29. [\[CrossRef\]](#)
2. Bunkhumpornpat, C.; Sinapiromsaran, K.; Lursinsap, C. Safe-level-SMOTE: Safe-level-synthetic minority over-sampling Technique for handling the class imbalanced problem. In *Lecture Notes in Computer Science. Advances in Knowledge Discovery and Data Mining*; Springer: Berlin/Heidelberg, Germany, 2009; pp. 475–482. [\[CrossRef\]](#)
3. Kaur, H.; Singh Pannu, H.; Malhi, A.K. A systematic review on imbalanced data challenges in machine learning: Applications and solutions. *ACM Comput. Surv.* **2019**, *52*, 1–36. [\[CrossRef\]](#)
4. Branco, P.; Torgo, L.; Ribeiro, R.P. A survey of predictive modeling on imbalanced domains. *ACM Comput. Surv.* **2016**, *49*, 1–50. [\[CrossRef\]](#)
5. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: Synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **2002**, *16*, 321–357. [\[CrossRef\]](#)
6. Zheng, Z.; Cai, Y.; Li, Y. Oversampling method for imbalanced classification. *Comput. Inform.* **2015**, *34*, 1017–1037.
7. Zhang, Z.L.; Peng, R.R.; Ruan, Y.P.; Wu, J.; Luo, X.G. ESMOTE: An overproduce-and-choose synthetic examples generation strategy based on evolutionary computation. *Neural Comput. Appl.* **2023**, *35*, 6891–6977. [\[CrossRef\]](#)
8. Sáez, J.A.; Krawczyk, B.; Woźniak, M. Analyzing the oversampling of different classes and types of examples in multi-class imbalanced datasets. *Pattern Recognit.* **2016**, *57*, 164–178. [\[CrossRef\]](#)
9. Liu, R. A novel synthetic minority oversampling technique based on relative and absolute densities for imbalanced classification. *Appl. Intell.* **2023**, *53*, 786–803. [\[CrossRef\]](#)
10. Vuttipittayamongkol, P.; Elyan, E. Neighbourhood-based undersampling approach for handling imbalanced and overlapped data. *Inf. Sci.* **2020**, *509*, 47–70. [\[CrossRef\]](#)
11. Rao, K.N.; Rao, T.V.; Lakshmi, D.R. A Novel Class Imbalance Learning using Ordering Points Clustering. *Int. J. Comput. Appl.* **2012**, *51*, 16.
12. Beckmann, M.; Ebecken, N.F.F.; Pires de Lima, B.S.L. A KNN Undersampling Approach for Data Balancing. *J. Intell. Learn. Syst. Appl.* **2015**, *7*, 104–116. [\[CrossRef\]](#)
13. Zhang, Y.; Kang, B.; Hooi, B.; Yan, S.; Feng, J. Deep long-tailed learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2023**, *45*, 10795–10816. [\[CrossRef\]](#) [\[PubMed\]](#)
14. Wei, C.; Sohn, K.; Mellina, C.; Yuille, A.; Yang, F. Crest: A class-rebalancing self-training framework for imbalanced semi-supervised learning. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 10857–10866.
15. Zang, Y.; Huang, C.; Loy, C.C. FASA: Feature augmentation and sampling adaptation for long-tailed instance segmentation. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, QC, Canada, 10–17 October 2021; pp. 3457–3466.
16. Yang, L.; Jiang, H.; Song, Q.; Guo, J. A survey on long-tailed visual recognition. *Int. J. Comput. Vis.* **2022**, *130*, 1837–1872. [\[CrossRef\]](#)
17. Zhang, P.; Li, X.; Hu, X.; Yang, J.; Zhang, L.; Wang, L.; Choi, Y.; Gao, J. Vinvl: Revisiting visual representations in vision-language models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 5579–5588.
18. Elyan, E.; Moreno-Garcia, C.F.; Jayne, C. CDSMOTE: Class decomposition and synthetic minority class oversampling technique for imbalanced-data classification. *Neural Comput. Appl.* **2021**, *33*, 2839–2851. [\[CrossRef\]](#)
19. Batista, G.E.; Prati, R.C.; Monard, M.C. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explor. Newsl.* **2004**, *6*, 20–29. [\[CrossRef\]](#)
20. Gong, J.; Kim, H. RHSBoost: Improving classification performance in imbalance data. *Comput. Stat. Data Anal.* **2017**, *111*, 1–13. [\[CrossRef\]](#)
21. Seiffert, C.; Khoshgoftaar, T.M.; Van Hulse, J.; Napolitano, A. RUSBoost: A hybrid approach to alleviating class imbalance. *IEEE Trans. Syst. Man Cybernetics. Part A Syst. Hum.* **2010**, *40*, 185–197. [\[CrossRef\]](#)

22. Sarmanova, A. *Veri Madenciliğindeki Sınıf Dengesizliği Sorununun Giderilmesi*; YTÜ Fen Bilimleri Enstitüsü: İstanbul, Türkiye, 2013.
23. Cao, L.; Zhai, Y. Imbalanced data classification based on a hybrid resampling SVM method. In Proceedings of the 2015 IEEE 12th Intl Conf on Ubiquitous Intelligence and Computing and 2015 IEEE 12th Intl Conf on Autonomic and Trusted Computing and 2015 IEEE 15th Intl Conf on Scalable Computing and Communications and Its Associated Workshops (UIC-ATC-ScalCom), Beijing, China, 10–14 August 2015; pp. 1533–1536. [\[CrossRef\]](#)
24. Yildirim, P.; Birant, D.; Alpyildiz, T. Data mining and machine learning in textile industry. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* **2018**, *8*, e1228. [\[CrossRef\]](#)
25. Boser, B.E.; Guyon, I.M.; Vapnik, V.N. A training algorithm for optimal margin classifiers. In Proceedings of the COLT92: 5th Annual Workshop on Computational Learning Theory, Pittsburgh, PA, USA, 27–29 July 1992. [\[CrossRef\]](#)
26. Vadood, M.; Ghorbani, V. Predicting the Hairiness of Cotton Rotor Spinning Yarns by Artificial Intelligence. *J. Text. Polym.* **2018**, *6*, 15–21.
27. Anami, B.S.; Elemmi, M.C. Comparative analysis of SVM and ANN classifiers for defective and non-defective fabric images classification. *J. Text. Inst.* **2022**, *113*, 1072–1082. [\[CrossRef\]](#)
28. Li, W.; Cheng, L. Yarn-dyed woven defect characterization and classification using combined features and support vector machine. *J. Text. Inst.* **2014**, *105*, 163–174. [\[CrossRef\]](#)
29. Ghosh, A.; Guha, T.; Bhar, R.B. Identification of handloom and powerloom fabrics using proximal support vector machines. *Indian J. Fibre Text. Res.* **2015**, *40*, 87–93.
30. Zhan, Z.; Zhou, J.; Xu, B. Fabric defect classification using prototypical network of few-shot learning algorithm. *Comput. Ind.* **2022**, *138*, 103628. [\[CrossRef\]](#)
31. Haleem, N.; Bustreo, M.; Del Bue, A. A computer vision based online quality control system for textile yarns. *Comput. Ind.* **2021**, *133*, 103550. [\[CrossRef\]](#)
32. Fix, E.; Hodges, J.L. *Discriminatory Analysis-Nonparametric Discrimination: Small Sample Performance*; Air University, USAF School of Aviation Medicine: Montgomery, AL, USA, 1952.
33. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32. [\[CrossRef\]](#)
34. Maglogiannis, I.G. *Emerging Artificial Intelligence Applications in Computer Engineering: Real Word AI Systems with Applications in Ehealth, HCI, Information Retrieval and Pervasive Technologies*; IOS Press: Amsterdam, The Netherlands, 2007; Volume 160.
35. Akin, P.; Terzi, Y. Comparison of unbalanced data methods for support vector machines. *Turk. Klin. J. Biostat.* **2021**, *13*, 138–146. [\[CrossRef\]](#)
36. McCulloch, U.S.; Pitts, V. *Logical Calculus of Ideas Relating to Nervous Activity*; Automata: Moscow, Russia, 1956.
37. Rumelhart, D.E.; Durbin, R.; Golden, R.; Chauvin, Y. Backpropagation: The basic theory. In *Backpropagation: Theory, Architectures and Applications*; Psychology Press: London, UK, 1995; pp. 1–34.
38. Haykin, S. *Neural Networks and Learning Machines*, 3/E; Pearson Education India: Bangalore, India, 2009.
39. Haixiang, G.; Yijing, L.; Shang, J.; Mingyun, G.; Yuanyue, H.; Bing, G. Learning from class-imbalanced data: Review of methods and applications. *Expert Syst. Appl.* **2017**, *73*, 220–239. [\[CrossRef\]](#)
40. Freund, Y.; Schapire, R.; Abe, N. A short introduction to boosting. *J. Jpn. Soc. Artif. Intell.* **1999**, *14*, 1612.
41. Schapire, R.E. *Empirical Inference: Festschrift in Honor of Vladimir N. Vapnik*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2013; pp. 37–52.
42. Rizwan, A.; Iqbal, N.; Ahmad, R.; Kim, D.H. WR-SVM model based on the margin radius approach for solving the minimum enclosing ball problem in support vector machine classification. *Appl. Sci.* **2021**, *11*, 4657. [\[CrossRef\]](#)
43. Balaban, M.; Erdal, E. *Veri Madenciliği ve Makine Öğrenmesi Temel Algoritmaları ve R Dili ile Uygulamalar*; Çağlayan Kitabevi: İstanbul, Türkiye, 2015.
44. R Core Team. *R: A Language and Environment for Statistical Computing*; R Foundation for Statistical Computing: Vienna, Austria, 2010.
45. Medvedieva, K.; Tosi, T.; Barbierato, E.; Gatti, A. Balancing the Scale: Data Augmentation Techniques for Improved Supervised Learning in Cyberattack Detection. *Eng* **2024**, *5*, 2170–2205. [\[CrossRef\]](#)
46. Dua, D.; Graff, C. UCI Machine Learning Repository. 2017. Available online: <https://archive.ics.uci.edu> (accessed on 10 October 2023).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.